

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Machhe, Belagavi, Karnataka - 590018



A Mini Project

Report On

“AI-Powered Gesture Driven Smart Slide Navigation System”

Submitted in partial fulfillment towards Mini Project (BIS586) of 5th Semester

**Bachelor of Engineering
in
Information Science and Engineering
*Submitted by***

Sana Anjum	[4GW23IS040]
Saranya G	[4GW23IS044]
Sneha Patel A V	[4GW23IS049]
Swathi S	[4GW23IS056]

Under the Guidance of

Dr. Chaya P

Associate Professor

Department of Information Science and Engineering
GSSSIETW, Mysuru



DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

(Accredited by NBA, New Delhi, Validity 01.07.2023 to 30.06.2026)

GSSS INSTITUTE OF ENGINEERING & TECHNOLOGY FOR WOMEN

(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi & Govt. of Karnataka)

K.R.S ROAD, METAGALLI, MYSURU-570016, KARNATAKA

Accredited by NAAC

2025-2026

Geetha Shishu Shikshana Sangha(R)

GSSS INSTITUTE OF ENGINEERING & TECHNOLOGY FOR WOMEN

K.R.S Road, Mysuru-570016, Karnataka

(Affiliated to VTU, Belagavi, Approved by AICTE -New Delhi & Govt. of Karnataka)

Accredited by NAAC

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

(Accredited by NBA, New Delhi, Validity: 01.07.2023– 30.06.2026)



CERTIFICATE

Certified that the 5th Semester Mini-Project (BIS586) titled “AI-Powered Gesture Driven Smart Slide Navigation System” is a bonafide work carried out by Sana Anjum (4GW23IS040), Saranya G (4GW23IS044), Sneha Patel A V (4GW23IS049) and Swathi S (4GW23IS056) in partial fulfilment for the award of degree of Bachelor of Engineering in Information Science & Engineering of the Visvesvaraya Technological University, Belagavi, during the year 2025-26. The Mini-Project report has been approved as it satisfies the academic requirements with respect to the Mini-project work prescribed for Bachelor of Engineering Degree.

Signature of the Guide

Dr. Chaya P

Signature of the HOD

Dr. Gururaj K S

ACKNOWLEDGEMENT

The joy and satisfaction that accompany the successful completion of any task would be incomplete without mentioning the people who made it possible.

First and foremost we offer our sincere phrases of thanks to **Late Smt. Vanaja B Pandit, Former Honorary Secretary, GSSS(R) Mysuru** for the blessings and support.

We offer our sincere thanks to **Smt. Anupama B Pandit, Secretary, GSSS (R) and the Management of GSSSIETW, Mysuru** for providing help and support to carry out the Mini-project.

We would like to express our gratitude to our **Principal, Dr. Shivakumar M** for providing us a congenial environment for engineering studies and also for having shown us the way to carry out the Mini-project.

We consider it a privilege and honour to express our sincere thanks to **Dr. Gururaj K S**, Professor and Head, Department of Information Science & Engineering for his support and invaluable guidance throughout the tenure of this Mini-project.

We would like to thank our **Guide Dr. Chaya P**, Associate Professor, Department of Information Science & Engineering for her support, guidance, motivation, encouragement for the successful completion of this Mini-project.

We would like to thank our Mini-Project Co-ordinator **Dr. Padmaja K**, Assistant Professor, Department of Information Science & Engineering for their constant monitoring, guidance & motivation throughout the tenure of this Mini-project.

We intend to thank all the teaching and non-teaching staff of our Information Science & Engineering department for their immense help and cooperation.

Finally, we would like to express our gratitude to our parents and friends who always stood with us to complete this work successfully.

Sana Anjum	(4GW23IS040)
Saranya G	(4GW23IS044)
Sneha Patel A V	(4GW23IS049)
Swathi S	(4GW23IS056)

ABSTRACT

Presentations are an essential medium for communication in educational, corporate, and professional environments. However, conventional presentation control methods relying on devices such as keyboards, mice, or remote clickers restrict a presenter's mobility, interrupt the natural delivery flow, and reduce audience engagement. The proposed AI-Powered Gesture-Driven Smart Slide Navigation System introduces an innovative hands-free approach to controlling presentations using Computer Vision and Machine Learning techniques. The system leverages MediaPipe and OpenCV libraries to capture and recognize real-time hand gestures via a standard webcam. Each recognized gesture is intelligently mapped to presentation commands such as next slide, previous slide, pointer display, and annotation mode, enabling seamless interaction without the need for physical input devices.

The methodology involves gesture detection through hand landmark extraction, preprocessing of video frames, and classification using defined gesture patterns. Recognized gestures are then integrated with slide control actions through PyAutoGUI, providing a natural and intuitive presentation experience. The system also supports customizable gesture mapping, allowing users to define personalized commands based on preference.

Experimental results demonstrate that the system achieves high accuracy in gesture detection under varying lighting and background conditions, ensuring real-time responsiveness and reliability. This project promotes accessibility for users with physical limitations, enhances audience engagement, and encourages the adoption of AI-based human-computer interaction in presentation systems.

TABLE OF CONTENTS

	Page No
Acknowledgement	i
Abstract	ii
Table of Contents	iii
List of Figure	iv
List of Tables	v
List of Snapshots	vi
CHAPTER 1: INTRODUCTION	1-3
1.1 Overview	1
1.2 Existing System	1
1.3 Aim and Objectives	2
1.4 Proposed System	2
1.5 Justification of the Project	3
1.6 Organization of the Report	3
CHAPTER 2 : REVIEW OF LITERATURE	4-9
2.1 Literature Survey	4
2.2 Summary of Literature Review	7
CHAPTER 3 : SYSTEM REQUIREMENTS AND DESIGN	14-16
3.1 Requirements	10
3.1.1 Functional Requirements	10
3.1.2 Non-Functional Requirements	10
3.1.3 Software Requirements	11
3.1.4 Hardware Requirements	11
3.2 System Design	12
3.2.1 System Architecture	12
3.3 Design Diagrams	13

3.3.1 Flow Diagram	13
3.3.2 Context Data Flow Diagram	15
CHAPTER 4 : IMPLEMENTATION	17-23
4.1 Code Snippets	17
CHAPTER 5 : TESTING	24-27
5.1 System Testing	24
5.2 Test Cases	26
CHAPTER 6 : RESULTS AND DISCUSSIONS	28-31
6.1 Snapshots	28
CONCLUSION & FUTURE ENHANCEMENT	32
REFERENCES	33-34

LIST OF FIGURES

FIGURE NUMBER	DESCRIPTION	PAGE NUMBER
3.1	System Architecture	12
3.2	Flow Diagram	14
3.3	Context Data Flow Diagram	16

LIST OF TABLES

TABLE NUMBER	DESCRIPTION	PAGE NUMBER
2.1	Literature Survey Summary	7
5.1	Test Cases	26

LIST OF SNAPSHOTS

TABLE NUMBER	DESCRIPTION	PAGE NUMBER
Snapshot 6.1	User Interface for the system	28
Snapshot 6.2	Upload Function	28
Snapshot 6.3	Gesture for Next	29
Snapshot 6.4	Gesture for Previous	29
Snapshot 6.5	Gesture for pointer	30
Snapshot 6.6	Gesture for Annotation	30
Snapshot 6.7	Gesture for Erasing	31
Snapshot 6.8	Customization Tab	31

CHAPTER 1

INTRODUCTION

1.1 Overview

Presentations play a vital role in academic, corporate, and professional communication, allowing individuals to effectively convey information and ideas. Despite advancements in digital tools, the method of controlling presentations has remained largely unchanged, relying on traditional hardware devices such as keyboards, mice, or remote clickers. These manual control methods often restrict the presenter's mobility, interrupt the natural flow of delivery, and divert attention from the audience to the control device, resulting in a less engaging experience.

The AI-Powered Gesture-Driven Smart Slide Navigation System aims to overcome these limitations by providing a hands-free, intelligent presentation control mechanism. The system utilizes Computer Vision and Artificial Intelligence (AI) techniques to detect and interpret hand gestures in real time using a standard webcam. Through libraries such as MediaPipe and OpenCV, the system tracks hand landmarks, identifies gesture patterns, and maps them to specific presentation actions like moving to the next slide, returning to the previous slide, activating the pointer, drawing, or erasing annotations.

This project not only enhances the fluidity and intuitiveness of presentations but also promotes user accessibility, particularly for individuals with physical challenges. The model achieves this by eliminating dependency on hardware devices, enabling a more natural and interactive mode of communication. The system's architecture emphasizes modularity, real-time gesture processing, and seamless integration with existing presentation software. It operates locally without requiring an internet connection, ensuring privacy and security of user data.

In essence, this project demonstrates the effective application of AI and Computer Vision to improve human-computer interaction, transforming traditional presentation control into a smart, gesture-based system that enhances user experience, efficiency, and engagement.

1.2 Existing System

In traditional presentation systems, users depend on external hardware devices such as keyboards, mice, or wireless clickers to navigate slides or control on-screen elements. Although these tools have been widely used for years, they introduce several limitations that affect the overall presentation

experience. The need for manual input restricts the presenter's movement, forcing them to stay near the device or computer system to change slides or interact with the presentation. This dependence on hardware disrupts the natural flow of communication, making it difficult for the presenter to maintain continuous engagement with the audience. It often leads to moments of distraction when the presenter must pause to locate a button or adjust a device. Additionally, hardware-based control systems are prone to connectivity issues such as signal loss, battery failure, or sensor interference, further interrupting the presentation. The existing systems also fail to address accessibility challenges faced by individuals with physical disabilities, as they rely on manual control mechanisms. Furthermore, these methods lack adaptability and do not provide intelligent or automated features to enhance the overall presentation quality.

In summary, the existing presentation control systems are static, hardware-dependent, and non-intuitive, highlighting the need for an advanced, user-friendly, and intelligent solution that allows presenters to interact naturally with their slides without relying on physical devices.

1.3 Aim & Objectives

The aim of this project is to design and develop a real-time gesture-controlled presentation system that leverages computer vision and hand-tracking technologies to enable seamless slide navigation, customizable gesture-to-action mapping, and interactive control through an intuitive interface.

- To capture real-time hand gestures through a laptop webcam and process them using OpenCV and MediaPipe for accurate recognition.
- To map recognized gestures into presentation controls such as slide navigation, pointer display, drawing, and erasing.
- To provide flexibility by allowing users to customize gesture-to-action mappings effectively.
- To design a user-friendly interface with live webcam overlay, performance metrics, and testing under varied conditions.

1.4 Proposed System

The proposed system replaces traditional hardware-based presentation control with an AI-powered, gesture-driven solution that allows presenters to navigate slides and perform actions through real-time hand gesture recognition using a webcam. By integrating MediaPipe, OpenCV, and PyAutoGUI, the system detects and interprets gestures such as next slide, previous slide, pointer activation, and drawing without the need for any physical device. This contactless approach enhances mobility, engagement, and accessibility, offering a seamless, intuitive, and efficient method of controlling presentations in both academic.

1.5 Justification of the Project

- **Enhances Accessibility:** Enables hands-free control of presentations, benefiting individuals with physical challenges or mobility limitations.
- **Improves Engagement:** Allows presenters to move freely and maintain continuous interaction with the audience without being restricted by devices.
- **Promotes Innovation:** Integrates artificial intelligence and computer vision for smart, intuitive, and real-time gesture recognition.
- **Increases Efficiency:** Eliminates interruptions caused by manual control devices, ensuring a smooth and uninterrupted presentation flow.
- **Cost-Effective Solution:** Removes the need for additional hardware such as remotes or sensors, reducing setup costs and maintenance.
- **User-Centric Design:** Provides customizable gesture mapping and a user-friendly interface suitable for academic, professional, and corporate environments.

1.6 Organization of the Report

The report is organized in various chapters and the brief description of each chapter is given below.

Chapter 1: Introduction – Provides an overview of the project, including discussions on the existing system, proposed system, aim and objectives, and justification of the project.

Chapter 2: Literature Survey – Presents the related works, research studies, and advancements in the field of gesture recognition, computer vision, and AI-based human–computer interaction.

Chapter 3: System Design and Requirements – Describes the functional and non-functional requirements, system architecture, and various design diagrams such as data flow diagrams and flow charts.

Chapter 4: Implementation Methodology – Explains the technical approach, algorithmic flow, libraries used, and integration of gesture recognition with the presentation control system.

Chapter 5: Testing and Validation – Covers different test cases, execution procedures, and evaluation of system performance and accuracy under varying conditions.

Chapter 6: Results and Discussion – Presents the system outputs, snapshots, followed by the conclusion, future enhancements, and references.

CHAPTER 2

REVIEW OF LITERATURE

2.1 Literature Survey

This chapter focuses on the literature survey conducted as part of this project. A literature survey is a discursive analysis rather than a mere summary or list of individual studies. It is an iterative process involving the evaluation and synthesis of information. The primary objective is to explore unresolved issues or gaps in the existing knowledge. A well-executed literature survey provides a foundation for developing an actionable plan for empirical investigation and analysis.

[1] Gesture Driven Presentation Control

This paper by D. Sirisha discusses a real-time interactive presentation control system using Mediapipe and OpenCV. It highlights the limitations of supporting only a limited set of gestures and being restricted to specific predefined commands. The study identifies the potential to expand gesture variety and improve system accuracy through optimized model training.

[2] Enhancing Presentation Dynamics with Real-Time Hand Gesture Recognition

Divya Upadhyay reviews existing methods and algorithms used for gesture-based presentation systems. The paper emphasizes the role of AI in improving interactivity but remains theoretical, lacking practical implementation or experimental results. It highlights a major research gap in developing deployable real-time systems.

[3] A Machine Learning Based Approach for Dynamic Hand Gesture Recognition

Arpneek Kaur and Sandhya Bansal propose a machine learning model for dynamic gesture recognition in human–robot interaction scenarios. The study demonstrates good classification accuracy but requires large datasets and exhibits noticeable delays in real-time performance.

[4] A Continuous Real-Time Hand Gesture Recognition Method Based on Skeleton

Thi-Lan Le presents a skeleton-based hand gesture recognition method using a temporal deep network (TD-Net). The approach provides efficient gesture detection but is limited to skeleton input and faces reduced accuracy in complex or cluttered backgrounds.

[5] AI-Based Computer Vision Techniques and Expert Systems

Yasunari Matsuzaka and Ryu Yashiro review the use of AI-driven computer vision and expert systems across human–computer interaction and medical applications. While comprehensive, the paper lacks experimental validation and does not address gesture control in presentation systems specifically.

[6] Hand Gesture Recognition Using Convolutional Neural Networks

Vijay Madaan and Neha Sharma introduce a CNN-based approach achieving high accuracy in gesture classification. Despite promising results, the model lacks real-time testing and can be further improved by implementing live detection for presentation applications.

[7] Hand Sign and Gesture Recognition System

Priyal Raj and co-authors design a vision-based gesture recognition system for sign language communication. Although effective, it is limited to specific sign gestures and requires extension to broader use cases like presentation control.

[8] Hand Gesture-Based AI Controller for Presentation, Virtual Drawing, and System Volume Management

Akshay Kumar and his team develop a system that integrates AI-based gesture control for multiple operations including presentations and virtual drawing. While functional, the system is complex and experiences increased error rates during prolonged use, requiring GPU acceleration for stability.

[9] Gesture-Based Model Using Deep Learning

Anuja Dixit and her team propose a deep learning model for gesture-based slide control. The system demonstrates accuracy improvements through neural networks but still requires real-world validation and enhanced robustness under different conditions.

[10] Efficient Gesture-Based Presentation Controller Using Transfer Learning Algorithm

Shaik Faiz Ahamed and his team implement transfer learning techniques for gesture-based presentation control. The approach effectively reuses pretrained models but demands high computational resources and lacks optimization for lightweight, real-time applications.

[11] Dynamic Hand Gesture Recognition

Subash Chandra Bose Jaganathan and co-authors focus on dynamic gesture recognition using machine learning in real time. The study achieves good classification accuracy but requires optimization for speed and dataset efficiency when applied to presentation systems.

[12] Smart Presentation System Using Hand Gestures

Bhairavi Pustode and her team develop a smart presentation system using computer vision for controlling slides. The prototype performs well but has limited hardware testing, suggesting future improvements in real-time robustness and environmental adaptability.

[13] Controlling PowerPoint Presentation Using Hand Gestures in Real-Time

Ch. Sai Charan and his co-authors design a PowerPoint control system combining gesture recognition and speech-to-text. The model performs well in standard conditions but shows reduced accuracy in poor lighting and with complex gestures.

[14] Automated Digital Presentation Control Using Hand Gesture Technique

Salonee Powar and her team propose a gesture-based controller for digital presentations. The system recognizes a few gestures accurately but suffers from environmental sensitivity, especially under variable lighting conditions.

[15] Hand Gesture Recognition Based on Computer Vision

Munir Oudah and colleagues present a comprehensive review of computer vision techniques for gesture recognition. The paper identifies multiple approaches applicable to real-time presentation systems and highlights the need for a practical, optimized, and robust model for real-world use.

2.2 Summary of Literature Review

Table 2.1: Literature Survey Summary

No	Title	Author(s)	Description	References	Year	Limitations
1	Gesture Driven Presentation Control	D. Sirisha	Real-time interactive control using Mediapipe& OpenCV	IEEE	2024	Limited gestures, restricted to specific commands, can expand gesture set and improve accuracy.
2	Enhancing Presentation Dynamics with Real-time Hand Gesture Recognition	Divya Upadhyay	Review of methods for gesture-based presentation	IEEE Access	2024	Review only,lacks experiments, highlights research gap in practical implementation.
3	A Machine Learning Based Approach for Dynamic Hand Gesture Recognition in Human-Robot Interaction	Arpneek Kaur, Sandhya Bansal	ML-based dynamic gesture recognition for human-robot interaction	IEEE	2024	Requires large datasets, delay in real-time interaction; focused on robots, not presentations.

4	A Continuous Real-time Hand Gesture Recognition Method based on Skeleton	Thi-Lan Le	Skeleton-based recognition system using TD-Net for detection & recognition	IEEE	2024	Limited to skeleton-based input, performance may degrade in complex backgrounds, can be extended to slide navigation.
5	AI-Based Computer Vision Techniques and Expert Systems	Yasunari Matsuzaka, Ryu Yashiro	Review of AI-based CV & expert systems in medical and HCI	MDPI Sensors	2023	Mostly review, lacks experimental validation, gap in presentation-specific application.
6	Hand Gesture Recognition Using Convolutional Neural Networks	Vijay Madaan, Neha Sharma	CNN-based gesture recognition.	Chitkara University	2023	High dataset accuracy, lacks real-time validation, can implement real-time detection.
7	Hand Sign & Gesture Recognition System	Priyal Raj et al.	Recognition of signs & gestures for communication	IEEE Access	2023	Needs extension beyond sign language, can adapt model for presentation gestures.
8	Hand Gesture based AI Controller for Presentation, Virtual Drawing	Akshay Kumar et al.	Gesture recognition for multiple controls	Amrita Vishwa	2023	Complex,error margin increases, GPU needed, can focus on presentation control with simplified gestures.
9	Gesture Based Model using Deep Learning	Anuja Dixit et al.	DL model for gesture-based slide control	IEEE	2023	Needs real-world testing, can extend with better datasets.

10	Efficient Gesture-Based Presentation Controller Using Transfer Learning Algorithm	Shaik Faiz Ahamed et al.	Transfer learning for gesture control	ICCCI	2023	Limited to pretrained models, high compute required; gap in lightweight model for real-time presentations.
11	Dynamic Hand Gesture Recognition	Subash Chandra Bose Jaganathan et al.	ML-based dynamic gesture recognition in real-time	VIT Bhopal	2023	Needs optimization for speed & dataset, focused on gestures, can be adapted for presentation commands.
12	Smart Presentation System Using Hand Gestures	Bhairavi Pustode et al.	Smart presentation system using CV for slide control	Research Square	2023	Early-stage, limited hardware testing; project can extend to robust, real-time deployment.
13	Controlling PowerPoint Presentation using Hand Gestures in Real-Time	Ch Sai Charan, K Meenakshi, V Bhavani Reddy, V Kashyap	Control PowerPoint using gestures	IEEE Access	2023	Limited scope, accuracy drops in poor lighting; can improve real-time accuracy and gesture variety.
14	Automated Digital Presentation Control using Hand Gesture Technique	Salonee Powar et al.	Gesture-based PowerPoint controller	ICACC	2022	Limited gestures, lighting affects accuracy, can improve robustness & gesture variety.
15	Hand Gesture Recognition	Munir Oudah et al.	Review of CV techniques for gesture recognition	MDPI Sensors	2020	Review only, lacks practical system, identifies methods for use in project .

CHAPTER 3

SYSTEM REQUIREMENTS AND DESIGN

3.1 Requirements

3.1.1 Functional Requirements

The functional requirements focus on integrating real-time gesture detection, classification, and presentation control to ensure seamless slide navigation and enhanced user interaction. The system leverages computer vision, machine learning, and automation libraries to recognize gestures and perform presentation operations accurately and efficiently.

- **Real-Time Gesture Detection:** Capture live video feed using the system webcam and detect hand landmarks with MediaPipe for accurate tracking and recognition.
- **Gesture Classification:** Identify and classify detected gestures such as next slide, previous slide, pointer activation, drawing, and erasing based on predefined gesture patterns.
- **Action Execution:** Map each recognized gesture to its corresponding presentation control command using PyAutoGUI to ensure seamless navigation.
- **Customizable Gestures:** Allow users to define and modify gesture-to-action mappings according to personal preferences and usage patterns.
- **Live Interface Display:** Provide a real-time webcam overlay that visually displays detected gestures and current system status for user clarity.
- **Performance Monitoring:** Continuously track and log gesture recognition accuracy, response time, and user actions for testing and evaluation.
- **Error Handling:** Detect unrecognized or ambiguous gestures and prevent accidental slide transitions or misinterpretations during live use.

3.1.2 Non- Functional Requirements

The non-functional requirements define the quality attributes, performance standards, and operational constraints that ensure the system functions effectively and efficiently. These parameters enhance usability, maintainability, and overall user satisfaction of the AI-Powered Gesture-Driven Smart Slide Navigation System.

- **Performance:** The system must process gestures in real time with high accuracy, ensuring minimal delay and precise command execution during presentations.

- **Usability:** The interface should be simple, intuitive, and accessible to all users, enabling easy setup and operation without technical expertise.
- **Reliability:** The system should function consistently under varying lighting conditions and backgrounds without frequent errors or interruptions.
- **Security:** All operations should be performed locally without requiring internet connectivity, ensuring data safety and user privacy.
- **Scalability:** The system should support the addition of new gestures and features with minimal code modification and allow easy maintenance or updates.

3.1.3 Software Requirements

The software requirements for the AI-Powered Gesture-Driven Smart Slide Navigation System include Python as the primary programming language, along with computer vision and automation libraries for gesture recognition and control. OpenCV and MediaPipe are used for image processing and gesture tracking, while PyAutoGUI enables system-level interaction with presentation tools. These components together ensure accurate detection, efficient processing, and smooth real-time slide navigation.

- Operating System: Windows 10
- IDE: Visual Studio Code
- Programming Language: Python
- Libraries and Frameworks: OpenCV, MediaPipe, PyAutoGUI.

3.1.4 Hardware Requirements

The hardware requirements for the AI-Powered Gesture-Driven Smart Slide Navigation System ensure smooth real-time gesture detection, image processing, and system performance. With adequate processing power and memory, the system delivers reliable and responsive slide navigation using computer vision and AI-based controls.

- Processor: Intel i3 or higher
- Processor Speed: 2.5 GHz or above
- RAM: Minimum 8 GB
- Hard Disk Space: 512 GB or more
- Webcam: Built-in or external HD camera for gesture detection

3.2 System Design

3.2.1 System Architecture

Figure 3.1, shows the system is designed to enhance the efficiency and interactivity of presentations by allowing users to control slides through hand gestures instead of conventional input devices. Traditional presentation systems rely on hardware such as keyboards or clickers, which limit mobility and disrupt the presenter's flow. The AI-Powered Gesture-Driven Smart Slide Navigation System utilizes computer vision and machine learning techniques to recognize real-time hand gestures, enabling hands-free control, improved engagement, and accessibility for users. The system aims to provide a seamless, intelligent, and user-friendly solution for dynamic presentation management.

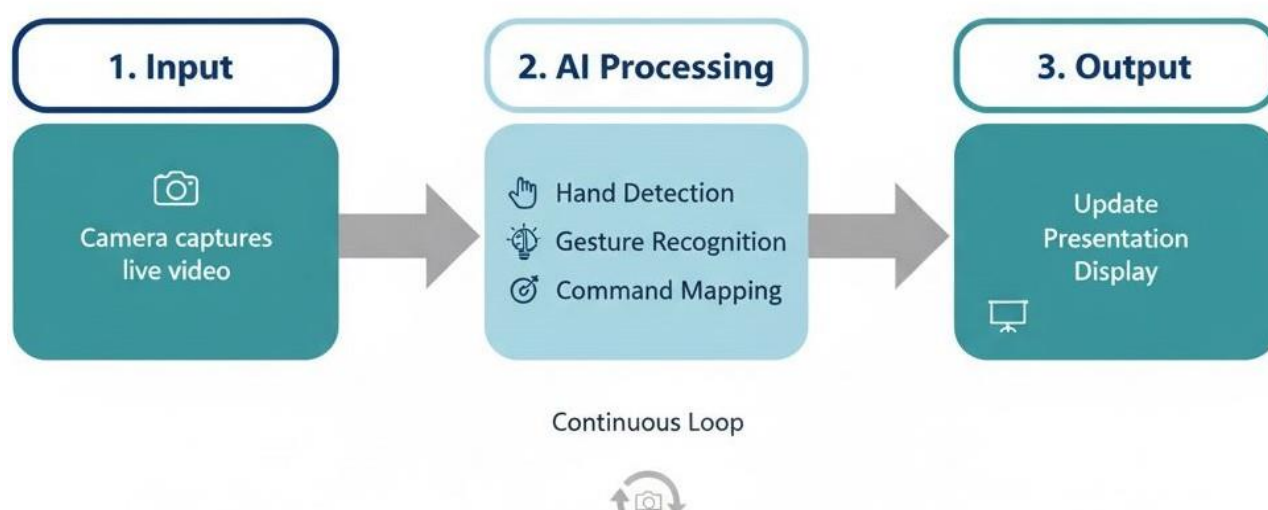


Figure 3.1: System Architecture

- **Input Module**

The Input Module captures real-time video frames using the system's webcam. These frames contain the presenter's hand gestures, which form the raw data for gesture recognition. The module ensures continuous frame capture at an optimal rate to maintain smooth system responsiveness.

- **Preprocessing and Hand Tracking Module**

This module preprocesses the captured frames using OpenCV to enhance image quality and remove noise. MediaPipe is employed to detect and extract 3D hand landmarks from each frame. The hand tracking ensures precise recognition of gesture patterns, regardless of background or lighting variations.

- **Gesture Recognition Module**

The Gesture Recognition Module analyzes hand landmark coordinates and classifies gestures into

predefined categories such as next slide, previous slide, pointer activation, drawing, and erasing. It uses threshold-based logic and geometric relationships between hand joints to ensure accurate classification in real time.

- **Action Mapping Module**

Once the gesture is recognized, the Action Mapping Module translates it into corresponding keyboard or mouse actions using PyAutoGUI. For example, a right-hand swipe may trigger the “next slide” command, while a left-hand swipe returns to the previous slide. This module bridges gesture input with presentation control.

- **Output Interface Module**

The Output Interface Module executes the mapped commands on the presentation software such as Microsoft PowerPoint, Google Slides, or PDF viewers. It provides a smooth transition between slides and displays visual feedback to confirm the action performed.

3.3 Design Diagrams

3.3.1 Flow Diagram

Figure 3.2 shows the Flow Diagram that illustrates how information moves through the AI-Powered Gesture-Driven Smart Slide Navigation System. The process begins with capturing real-time video input from the webcam, which is then passed through several preprocessing steps such as frame resizing, noise reduction, and hand landmark extraction. After preprocessing, the data enters the gesture detection and classification module, where trained models accurately recognize the user’s hand gestures and map them to predefined commands. These commands are then forwarded to the slide control and interaction engine, which performs actions like changing slides, enabling pointer mode, or activating the drawing tool. The system continuously loops through this flow to provide real-time responsiveness. Each classified gesture is then mapped to a corresponding system command and forwarded to the slide control and interaction engine, which executes actions like moving to the next slide, going back, activating pointer mode, or enabling drawing tools. Additionally, the system provides visual or on-screen cues to inform the user about the detected gesture and the action performed, enhancing usability. The entire workflow operates in a continuous, real-time loop, enabling smooth, responsive, and interruption-free presentation control. This seamless integration of modules ensures that users can interact naturally and confidently without relying on physical devices.

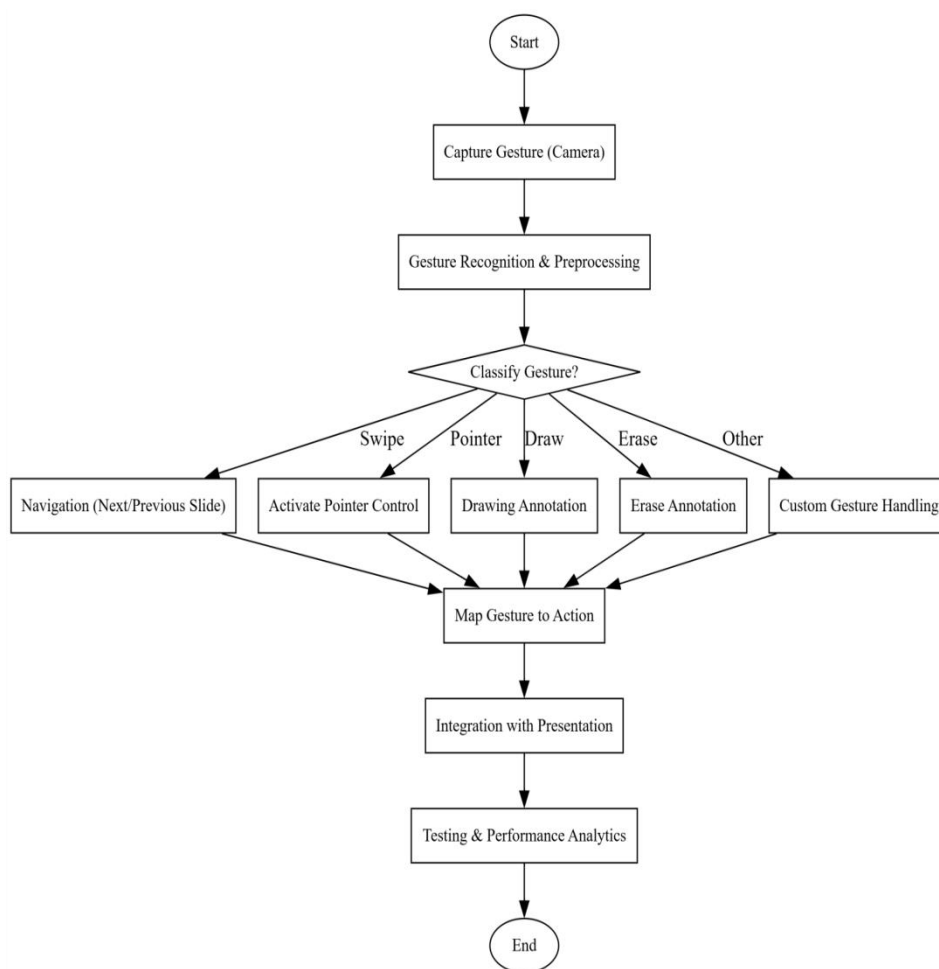


Figure 3.2: Flow Diagram

- **Capture Hand Gestures**

The process begins with the webcam capturing live video input of the user's hand gestures. This input forms the raw data for the system. Each video frame contains continuous hand movements that are used to identify gesture patterns in real time.

- **Preprocessing Frames**

The captured frames undergo preprocessing using OpenCV. This step includes converting images to the required color format, removing noise, and adjusting brightness or contrast to improve visibility. The preprocessed frames ensure better accuracy during gesture detection even under varied lighting conditions.

- **Hand Landmark Detection**

Once the frames are preprocessed, MediaPipe is used to detect hand landmarks. The module identifies key points such as finger joints and palm positions, converting the raw image input into coordinate-based data. This structured representation simplifies the recognition of gesture patterns.

- **Gesture Classification**

The extracted landmarks are analyzed to identify specific gestures such as next slide, previous slide, pointer activation, drawing, or erasing. The system uses predefined thresholds and coordinate relationships between landmarks to accurately classify each gesture in real time.

- **Command Mapping**

After the gesture is classified, it is mapped to the corresponding presentation command using PyAutoGUI. This module converts the recognized gesture into a software-level action (for example, pressing the right arrow for “next slide” or left arrow for “previous slide”).

- **Slide Control Execution**

The mapped command is executed directly on the presentation software. The system ensures minimal delay between gesture detection and slide movement, providing a smooth and responsive user experience.

- **Visual Feedback Display**

The interface displays real-time visual feedback such as bounding boxes or gesture labels to confirm recognition accuracy. This helps the presenter stay aware of detected gestures and system status during the presentation.

3.3.2 Context Data Flow Diagram

The Context Level Data Flow Diagram, also known as Level 0 DFD, provides a high-level overview of the entire system. It represents the AI-Powered Gesture-Driven Smart Slide Navigation System as a single process and shows its interactions with external entities such as the user and presentation software. This diagram helps in understanding the flow of data between the system and its environment without focusing on internal details.

In this system, the user interacts with the webcam to perform gestures, which are captured and processed by the system. The processed gesture data is then used to control the slides in real time. The presentation software receives commands from the system and responds with corresponding slide transitions or actions. The presentation software receives these commands and responds by performing corresponding actions such as moving to the next slide, returning to the previous slide, or activating interactive features. This high-level diagram helps visualize how the system seamlessly connects user gestures with automated slide control, ensuring smooth communication between all components involved.

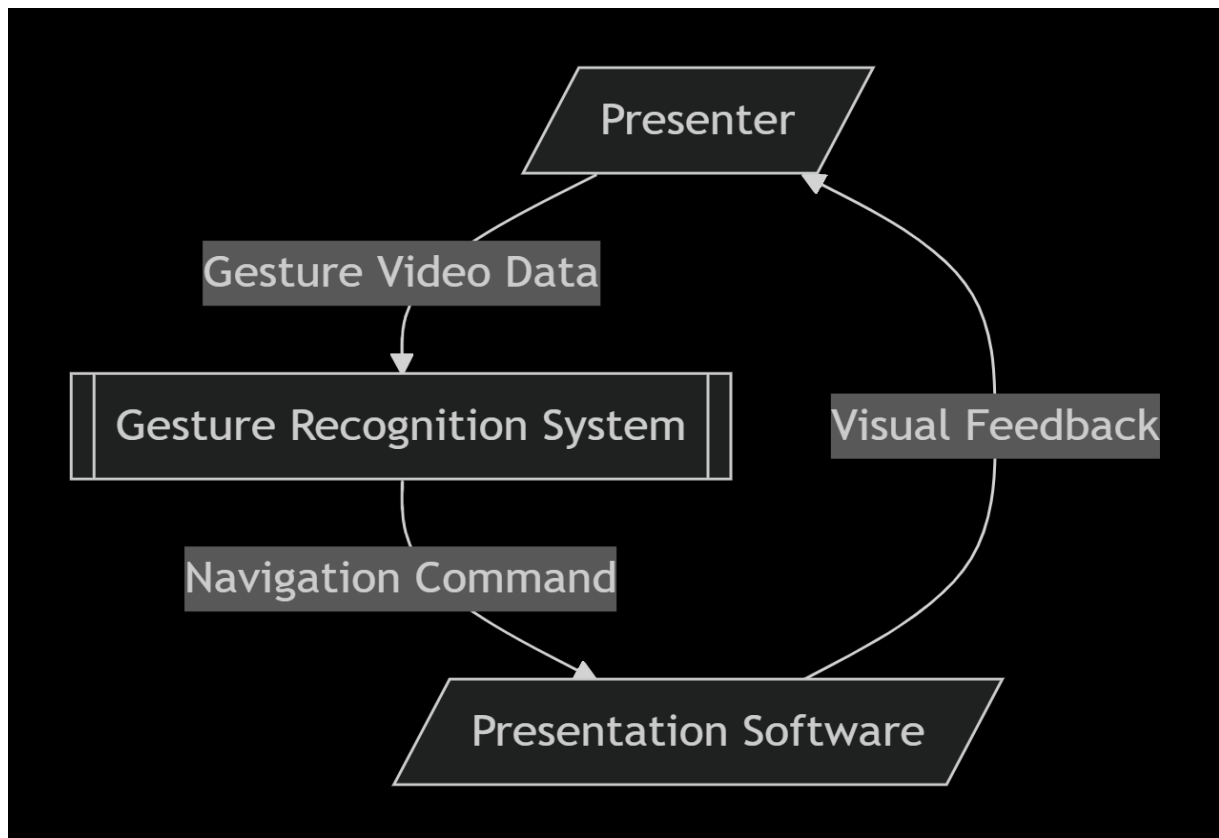


Figure 3.3: Context Data Flow Diagram

- **User**

The user acts as the primary external entity. They perform hand gestures such as swiping left, right, or pointing, which are captured by the webcam and processed by the system to control presentation slides.

- **Webcam Input**

The webcam captures live video input of the user's gestures and sends the raw image frames to the system for further processing.

- **Gesture Recognition System (Main Process)**

This represents the core process of the system. It receives the live video input, detects and classifies gestures using computer vision (MediaPipe and OpenCV), maps them to corresponding commands, and sends those commands to the presentation software.

- **Presentation**

This entity receives the mapped actions (like next or previous slide) from the system and executes them accordingly. It provides feedback in the form of slide transitions visible to the user.

CHAPTER 4

IMPLEMENTATION

The implementation phase focuses on transforming the design and theoretical concepts of the AI-Powered Gesture-Driven Smart Slide Navigation System into a fully functional, real-time application. The system enables presenters to control slides using simple hand gestures detected via a webcam, thereby eliminating the need for physical remotes or keyboards. This contactless interface enhances user convenience, accessibility, and presentation flow, particularly in educational and professional environments.

The project is implemented entirely in Python, leveraging its flexibility, extensive library support, and strong ecosystem for AI and computer vision applications. The system integrates multiple open-source frameworks such as OpenCV for video processing, MediaPipe for hand landmark detection, and PyAutoGUI for simulating user control inputs like keyboard presses and mouse movements. Each module is designed to perform a specific task, ensuring modularity, scalability, and ease of maintenance.

4.1 Code Snippets

➤ Hand Tracking

The Hand Detection Module is responsible for identifying the user's hand and extracting landmark coordinates in real time using the MediaPipe framework. MediaPipe Hands provides 21 key landmark points on the hand, allowing highly accurate tracking of finger and palm movements. This module preprocesses every video frame, detects the presence of hands, and returns a list of landmark positions which will be used by the gesturerecognition module. It also supports visualization by drawing the detected landmarks on the video feed. Overall, this module acts as the foundation of the entire system, enabling gesture-based interaction by accurately capturing hand structure and movement.

```
class HandDetector:
```

```
    def __init__(self, mode=False, maxHands=1, detectionCon=0.7, trackCon=0.6):
```

```
        self.mode = mode
```

```
        self.maxHands = maxHands
```

```
self.detectionCon = detectionCon

self.trackCon = trackCon

self.mpHands = mp.solutions.hands

self.hands = self.mpHands.Hands(

static_image_mode=self.mode,

max_num_hands=self.maxHands,

min_detection_confidence=self.detectionCon,

min_tracking_confidence=self.trackCon

)

self.mpDraw = mp.solutions.drawing_utils


def findHands(self, img, draw=True):

imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

self.results = self.hands.process(imgRGB)

if self.results.multi_hand_landmarks:

for handLms in self.results.multi_hand_landmarks:

if draw:

self.mpDraw.draw_landmarks(img, handLms, self.mpHands.HAND_CONNECTIONS)

return img


def findPosition(self, img, handNo=0):

lmList = []

if self.results.multi_hand_landmarks:
```

```
myHand = self.results.multi_hand_landmarks[handNo]

    for id, lm in enumerate(myHand.landmark):

        h, w, c = img.shape

        cx, cy = int(lm.x * w), int(lm.y * h)

lmList.append((id, cx, cy))

    return lmList
```

➤ **Gesture Recognition Module**

The Gesture Recognition Module interprets the hand landmark positions and converts them into meaningful gestures. Using predefined rules based on relative positions of thumb, index, and middle fingers, the module identifies gestures like Next Slide, Previous Slide, Pointer, and Idle. It continuously compares the detected gesture with the previous one to avoid repeated triggers. This rule-based gesture recognition makes the system lightweight, fast, and reliable without requiring machine learning training. The main objective of this module is to translate physical hand movements into high-level commands that the presentation controller can execute.

```
class GestureController:

    def __init__(self):
self.previous_gesture = None

    def recognizeGesture(self, lmList):

        if not lmList:
            return None

        # Thumb (id 4), Index (8), Middle (12), Ring (16), Pinky (20)
        index_tip = lmList[8][2]
        middle_tip = lmList[12][2]
        thumb_tip = lmList[4][1]
        index_base = lmList[5][1]

        if thumb_tip < index_base - 40:
```

```
gesture = "Next Slide"
elif thumb_tip > index_base + 40:
    gesture = "Previous Slide"
elif index_tip < middle_tip - 20:
    gesture = "Pointer"
else:
    gesture = "Idle"

self.previous_gesture = gesture
return gesture
```

➤ **Presentation Control Module**

This module is responsible for converting recognized gestures into real desktop actions using pyautogui. Once a gesture is classified, the Presentation Controller triggers keyboard or mouse events such as moving to the next slide, going back, or activating the on-screen pointer. It acts as the bridge between gesture inputs and actual presentation software, ensuring smooth and contactless slide navigation. This allows users to interact with applications like PowerPoint or Google Slides without touching any device, making the system suitable for classrooms, seminars, and touchless interfaces.

```
class PresentationController:
    def executeCommand(self, gesture):
        if gesture == "Next Slide":
            pyautogui.press('right')
        elif gesture == "Previous Slide":
            pyautogui.press('left')
        elif gesture == "Pointer":
            x, y = pyautogui.position()
            pyautogui.moveTo(x + 15, y)
        elif gesture == "Idle":
            pass
```

➤ Main Execution Logic

The Main Execution Logic integrates all modules and runs them continuously through the webcam stream. It performs the following operations in a loop.

1. Capture live video feed
2. Detect hands and landmarks
3. Recognize gestures
4. Execute actions based on gestures
5. Display real-time feedback on the screen

This acts as the control center of the project where all modules work together to form a complete AI-based Gesture Presentation System.

```
from HandTracker import HandDetector
from modules.GestureController import GestureController
from modules.PresentationController import PresentationController
```

```
cap = cv2.VideoCapture(0)
detector = HandDetector()
gesture_controller = GestureController()
presentation_controller = PresentationController()

while True:
    success, img = cap.read()
    if not success:
        break
    img = detector.findHands(img)
    lmList = detector.findPosition(img)
    gesture = gesture_controller.recognizeGesture(lmList)
    if gesture:
        presentation_controller.executeCommand(gesture)
        cv2.putText(img, f"Gesture: {gesture}", (10, 50),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
    cv2.imshow("AI Gesture Presentation System", img)
    if cv2.waitKey(1) & 0xFF == 27:
```

```
break
```

```
cap.release()  
cv2.destroyAllWindows()
```

➤ **Configuration (gesture_config.json)**

This JSON configuration file defines the customizable mapping between gestures and presentation actions. It allows the system to be flexible by enabling users to change which gestures correspond to actions like next slide, pointer activation, etc. By keeping gesture-action mapping in an external configuration file, the system becomes easy to update, scalable, and more user-friendly, as gesture rules can be extended without modifying the main code.

```
{  
  "next_slide": ["SwipeRight", "ThumbRight"],  
  "previous_slide": ["SwipeLeft", "ThumbLeft"],  
  "pointer": ["IndexUp"],  
  "start_presentation": ["ThumbsUp"],  
  "stop_presentation": ["Palm"]  
}
```

➤ **Logging Module**

The Logging Module stores every detected gesture along with its timestamp in a log file. This is important for

- Debugging
- System performance analysis
- Tracking user interactions
- Maintaining a usage history

By keeping a gesture log, developers can evaluate accuracy and recognize any misclassifications.

```
import datetime  
  
def log_gesture(gesture):  
    with open("gesture_log.txt", "a") as f:  
        timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
f.write(f"[{timestamp}] Gesture Detected: {gesture}\n")
```

This interface is designed to enhance presentation control and user experience through intuitive hand gestures, eliminating the need for physical input devices. By integrating real-time hand tracking and gesture recognition using computer vision, the system enables seamless slide navigation, pointer control, drawing, and erasing directly through gestures. The AI-powered model accurately identifies finger positions and interprets gesture patterns to perform corresponding slide operations, ensuring smooth interaction even in dynamic environments. The primary goal is to provide a touch-free, intelligent, and user-friendly solution that enhances productivity and accessibility during live presentations lectures.

CHAPTER 5

TESTING

5.1 System Testing

The AI-Powered Gesture-Driven Smart Slide Navigation System undergoes extensive testing to ensure its stability, responsiveness, and accuracy in detecting hand gestures for seamless presentation control. The testing phase validates the system's performance under various lighting conditions, gesture variations, and hardware configurations to guarantee a smooth and user-friendly experience during live presentations. The gesture detection accuracy of the system is fundamental to its performance, as the entire functionality depends on reliable hand tracking and gesture recognition.

To ensure this accuracy, the AI model is tested using various hand shapes, skin tones, backgrounds, and lighting environments. This helps validate whether gestures such as swipe, pointer, draw, and erase are consistently recognized without false triggers or missed inputs. By conducting these tests, the system demonstrates its ability to adapt to real-world variations and maintain stable detection. In addition to accuracy, the system's real-time responsiveness is evaluated to ensure that gestures translate into slide actions with minimal delay. The tests focus on measuring the latency between gesture execution and the corresponding response in the presentation. Smooth transitions, quick pointer movement, and instant visual feedback are essential to provide a natural and intuitive user experience. Ensuring low latency allows presenters to interact with their slides seamlessly without interruptions.

System integration and stability also play a key role in the overall performance of the gesture-controlled presentation system. Testing is performed to verify that all modules—hand tracking, gesture recognition, gesture mapping, and slide rendering—work together in synchronization. Prolonged testing sessions help ensure that the system does not experience crashes, lag, or loss of accuracy over time. This guarantees stable operation, even during extended presentations or classroom sessions. The graphical user interface (GUI), developed using Tkinter, undergoes usability testing to confirm that it is simple, accessible, and visually clear for all users. The interface is checked to ensure that users can upload PowerPoint files, configure gestures, and control presentation slides effortlessly without requiring technical knowledge. A well-organized layout and intuitive design contribute to making the system user-friendly and efficient.

➤ **Types of Testing**

● **Unit Testing**

Unit testing focuses on validating individual components of the gesture recognition system, such as the hand tracking module, gesture classification logic, slide control functions (Next, Previous, Pointer, Annotation, Erase), and system initialization. Each unit was tested independently to ensure correct functionality, accuracy, and robustness in real-time gesture processing.

● **Integration Testing**

Integration testing was conducted to verify that all modules—camera input, hand detection, gesture recognition, and slide navigation—work cohesively. This ensures that the output from the hand tracking module correctly triggers corresponding slide control actions within the presentation environment. The interactions between gesture classification and slide-control APIs were tested to detect communication or synchronization issues.

● **System Testing**

System testing validated the entire AI-Powered Gesture-Driven Smart Slide Navigation System as a unified platform. Both functional and non-functional aspects were tested to confirm that the system meets design specifications, provides smooth gesture response, and maintains compatibility with presentation software. End-to-end workflows from gesture input to slide execution were thoroughly examined. End-to-end workflows—from gesture input to slide execution—were thoroughly examined. Stress scenarios such as long-duration usage and high gesture frequency were also included to ensure system resilience.

● **Performance Testing**

Performance testing assessed the system's responsiveness, accuracy, and frame processing speed under different conditions, including varying lighting and gesture distances. The objective was to ensure the system remains stable and accurate during prolonged use without significant latency or frame drops. Additional evaluations were conducted to measure CPU/GPU utilization and memory consumption. The system was also tested under fast gesture transitions to confirm real-time detection without performance degradation.

● **User Acceptance Testing (UAT)**

User Acceptance Testing was carried out with presenters and faculty to evaluate usability, intuitiveness, and reliability in real-world scenarios. The goal was to ensure that gesture recognition, slide control, and customization features perform effectively during live presentations. Feedback was collected on ease of learning, comfort of use, and gesture sensitivity. Necessary refinements were made based on user suggestions to improve overall interaction quality and user satisfaction.

5.2 Test Cases

Table 5.1 : Test Cases

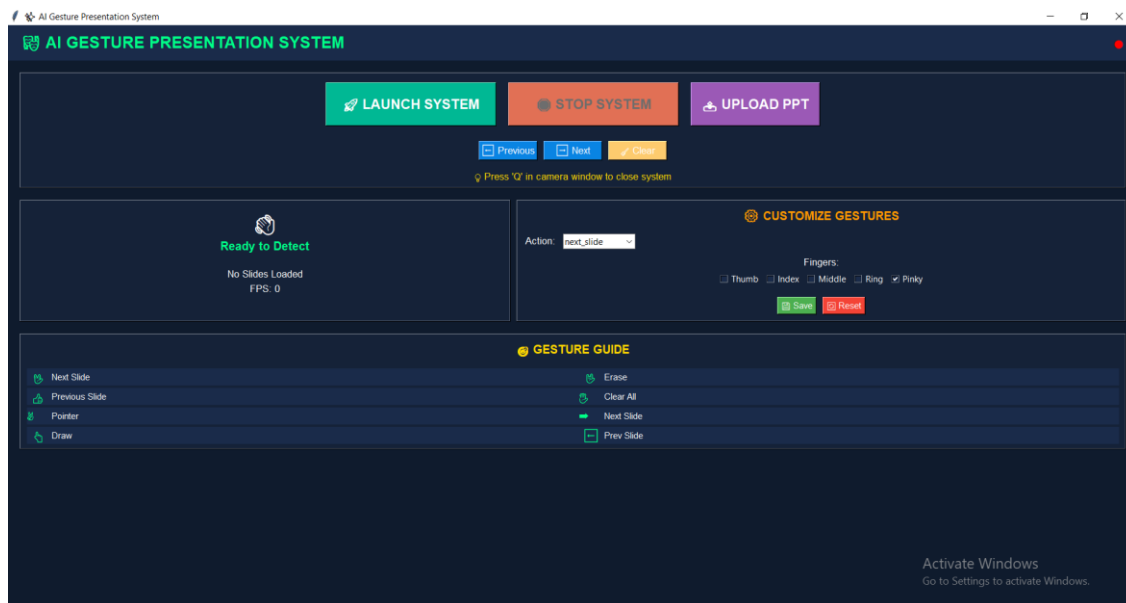
Case No.	Test Scenario	Expected Result	Actual Result	Status
1	Validation of gesture recognition module	The system should accurately detect and classify gestures such as Next, Previous, Pointer, Annotation, and Erase	Gestures correctly identified and executed	Pass
2	Verification of camera input acquisition	The system should capture hand movements seamlessly in real-time using the webcam	Camera input successfully acquired without latency	Pass
3	Evaluation of model response to invalid gestures	The system should ignore undefined or unrecognized gestures without triggering any slide action	Unrecognized gestures appropriately ignored	Pass
4	Functional testing of slide navigation	“Next” and “Previous” gestures should accurately transition between presentation slides	Slide navigation performed as intended	Pass
5	Validation of pointer control functionality	Pointer gesture should enable the virtual pointer with precise tracking on the slide	Pointer activated and tracked accurately	Pass
6	Verification of annotation feature	Annotation gesture should enable drawing mode for marking on slides	Annotation feature executed effectively	Pass
7	Evaluation of erase functionality	Erase gesture should remove existing annotations from the slide surface	Annotations cleared successfully	Pass

8	System initialization testing	Application should initialize the camera, gesture model, and interface without errors	System initialized and operational	Pass
9	Performance testing under prolonged operation	System should maintain consistent accuracy and responsiveness during continuous usage	System performance stable and reliable	Pass
10	User interface responsiveness testing	User interface should provide prompt feedback for control actions and calibration inputs	Interface response smooth and timely	Pass

CHAPTER 6

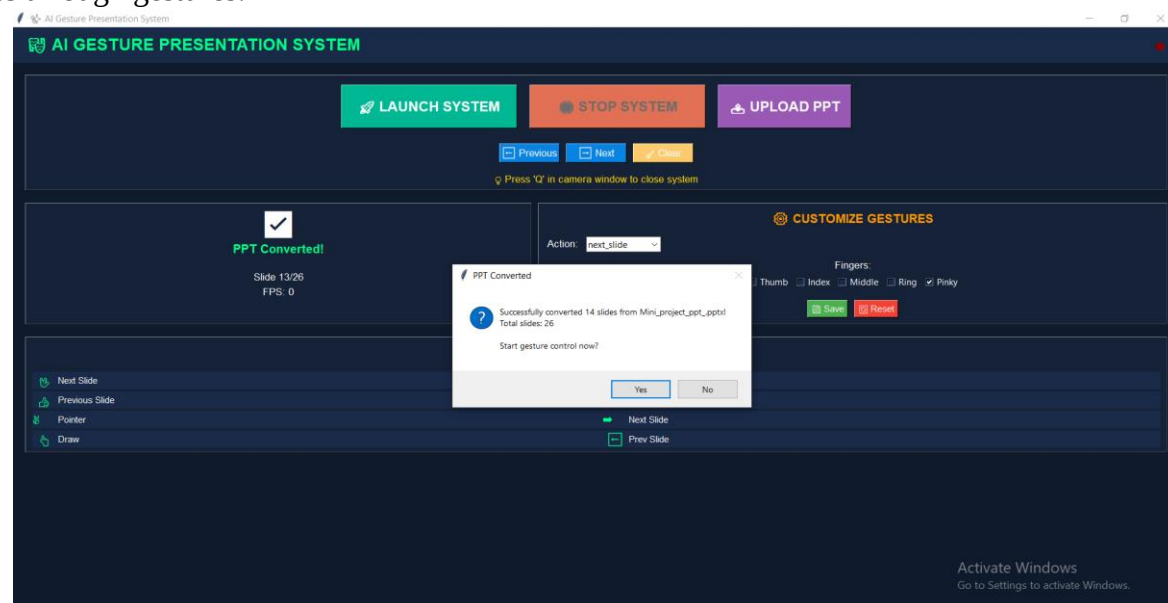
RESULTS AND DISCUSSIONS

6.1 Snapshots



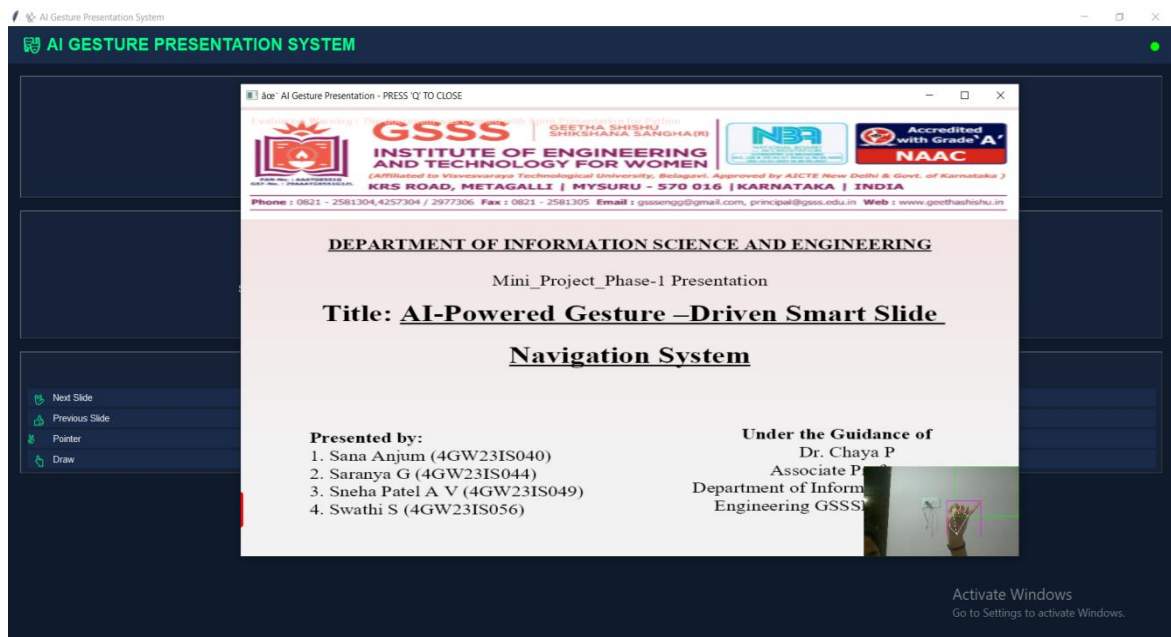
Snapshot 6.1: User Interface of AI Gesture Presentation System

Snapshot 6.1 shows the main user interface of the AI-Powered Gesture-Driven Smart Slide Navigation System developed using Tkinter. It allows users to upload presentations and control slides through gestures.



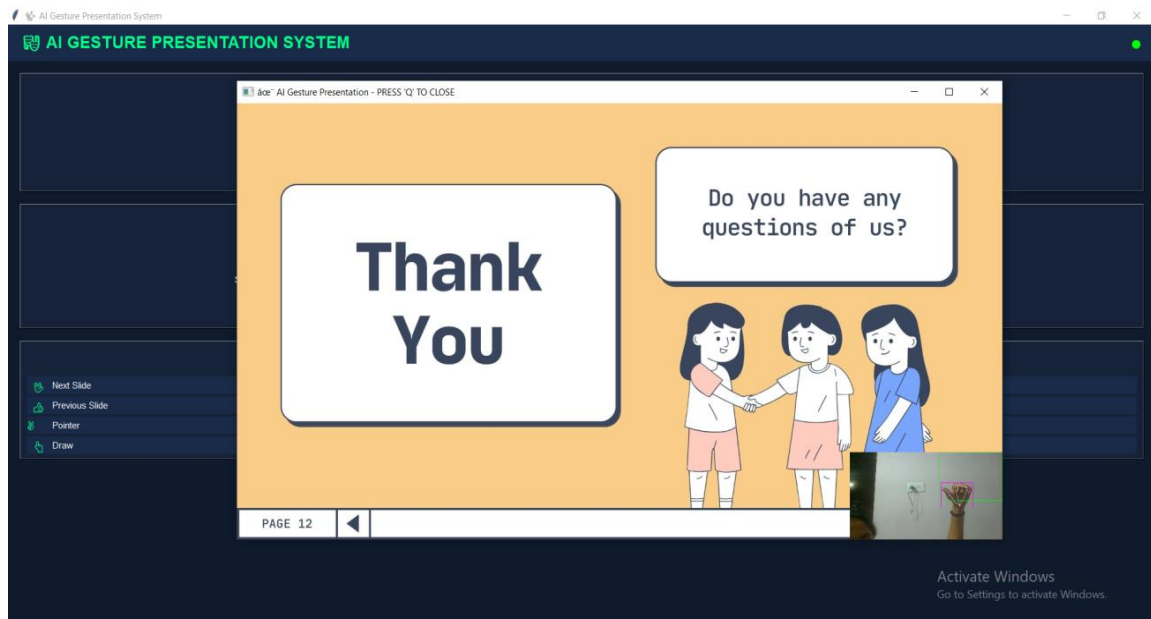
Snapshot 6.2: Upload PPT and Converted Dialog Box

Snapshot 6.2 displays the Upload PPT and Conversion Dialog Box, where the user selects a PowerPoint file for upload. The system automatically converts the slides into images and confirms successful conversion, allowing the user to start gesture-based control instantly.



Snapshot 6.3: Gesture for “Next Slide”

Snapshot 6.3 illustrates the gesture recognition for the “Next Slide” action in the AI-Powered Gesture-Driven Smart Slide Navigation System. When the user performs the predefined gesture, the system detects it through the camera and automatically advances to the next slide.



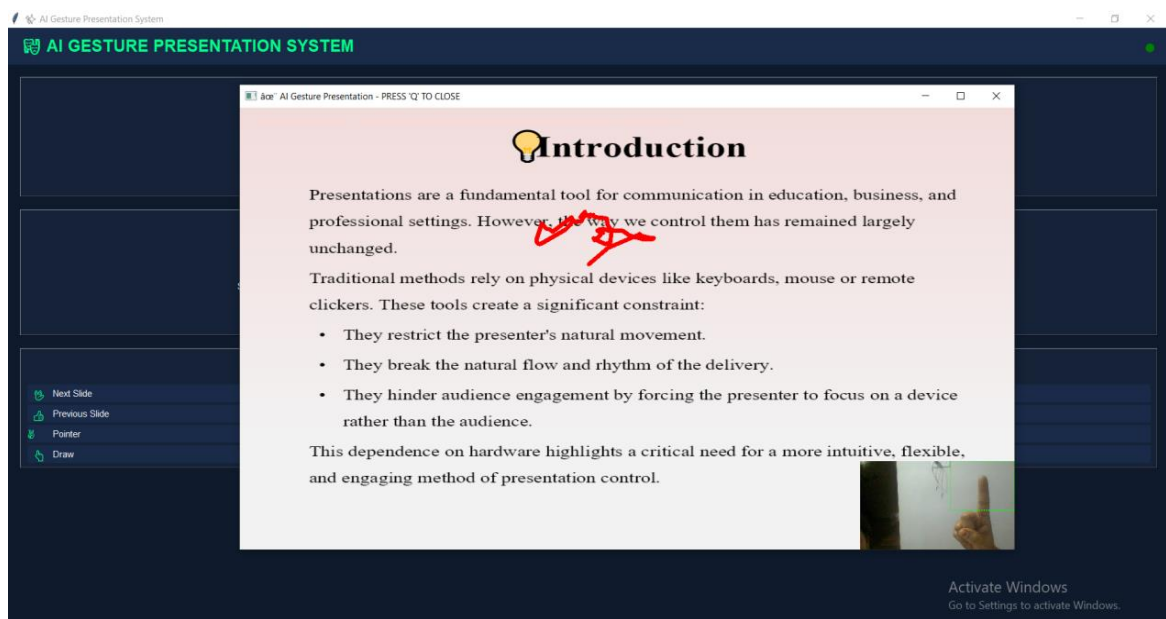
Snapshot 6.4: Gesture for “Previous Slide”.

Snapshot 6.4 shows the gesture detection for the “Previous Slide” action in the AI-Powered Gesture-Driven Smart Slide Navigation System. When the user performs the assigned gesture, the system accurately recognizes it and navigates back to the previous slide.



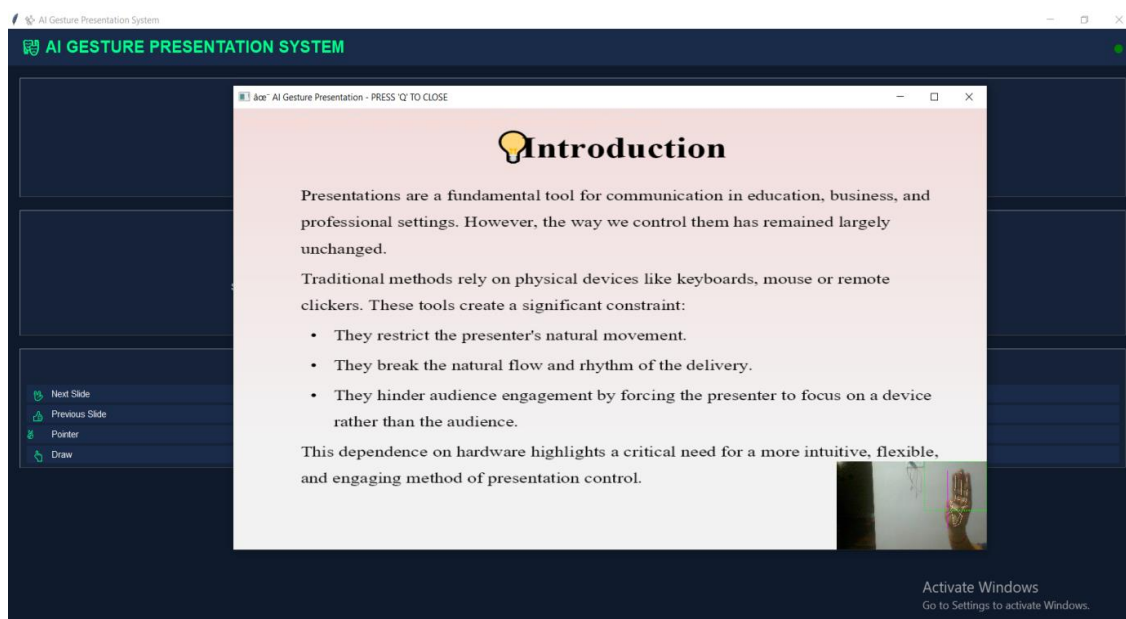
Snapshot 6.5: Gesture for “Pointer”

Snapshot 6.5 demonstrates the gesture recognition for the “Pointer” mode in the AI-Powered Gesture-Driven Smart Slide Navigation System. When the user performs the pointer gesture, a virtual red dot appears on the slide, following the hand’s movement in real time.



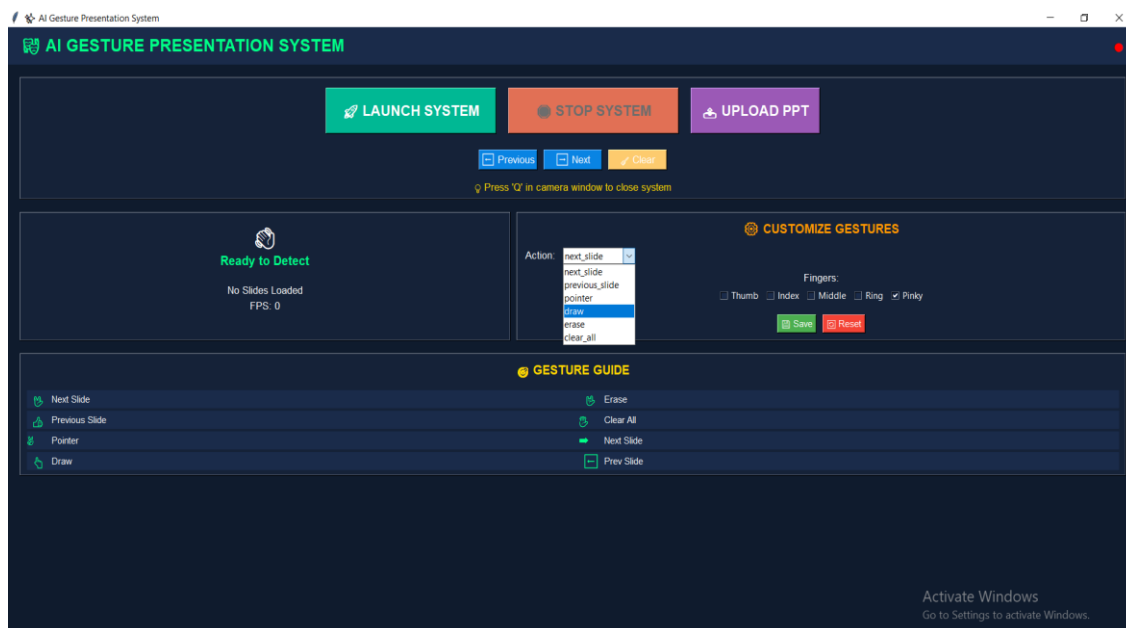
Snapshot 6.6: Gesture for “Annotation”

Snapshot 6.6 illustrates the gesture used for the “Annotation” mode in the AI-Powered Gesture-Driven Smart Slide Navigation System. When the user performs the designated gesture, the system switches to drawing mode, allowing real-time annotations directly on the slide. This feature helps presenters emphasize important points and make explanations more interactive.



Snapshot 6.7: Gesture For “Erasing”

Snapshot 6.7 depicts the gesture recognition for the “Erasing” function in the AI-Powered Gesture-Driven Smart Slide Navigation System. When the user performs the assigned erase gesture, the system detects it and removes the most recent annotation from the slide. This ensures quick corrections during presentations without interrupting the flow.



Snapshot 6.8: Customization Tab

Snapshot 6.8 shows the Customization Tab of the AI-Powered Gesture-Driven Smart Slide Navigation System. This section allows users to define and modify finger combinations for different gestures such as next slide, previous slide, draw, and erase. It provides flexibility for users to personalize gesture mappings according to their comfort and usage preferences.

CONCLUSION & FUTURE ENHANCEMENT

The AI-Powered Gesture-Driven Smart Slide Navigation System demonstrates an innovative approach to modern presentation control by enabling fully touch-free interaction through real-time hand gesture recognition. By seamlessly combining computer vision techniques, deep learning models, and an intuitive user interface, the system accurately identifies predefined gestures to execute commands such as moving between slides, activating pointer mode, drawing annotations, and erasing content. This significantly enhances the presenter's freedom and mobility by removing the dependency on physical devices like keyboards, remotes, or laser pointers. The system's modular architecture ensures streamlined integration of components such as hand detection, gesture classification, and presentation control, enabling efficient performance even in dynamic environments. The high accuracy and real-time responsiveness make the solution especially suitable for classrooms, conferences, corporate meetings, and interactive learning spaces. Overall, the project successfully achieves its goal of creating a smart, contactless, user-friendly presentation tool that elevates the presentation experience and aligns well with the modern shift toward AI-driven automation.

Future developments of the system can focus on enhancing its accuracy, usability, and overall flexibility. Features such as voice command integration, adaptive gesture learning, and multi-user gesture detection can make interactions more natural, personalized, and support collaborative presentations. Cloud-based slide storage and real-time synchronization would allow users to access their presentations from anywhere, improving portability and convenience. Expanding the system to mobile and web platforms would enable control from multiple devices, making it suitable for hybrid, online, and remote learning environments. Additional improvements, such as customizable gesture sets, context-aware commands, and AI-based predictive actions, could further increase efficiency and make the system highly intuitive. These enhancements would collectively make the system a versatile, intelligent, and comprehensive solution for modern presentation control.

REFERENCES

- [1] Gesture Driven Presentation Control ,D. Sirisha, G. S. Reddy, R. U. Sri, R. Hemanth, P. M. Kalyan ,Published in IJESAT ,2024

- [2] Enhancing Presentation Dynamics with Real-Time Hand Gesture Recognition: A Review , Divya Upadhyay, P. Sati, N. Jain , Published in IEEE Access , 2024

- [3] A Machine Learning Based Approach for Dynamic Hand Gesture Recognition in Human-Robot Interaction ,Arpneek Kaur, Sandhya Bansal ,Published in IEEE Robotics and Automation Letters , 2024

- [4] A Continuous Real-Time Hand Gesture Recognition Method Based on Skeleton ,Thi-Lan Le , Published in IEEE Sensors Journal , 2024

- [5] AI-Based Computer Vision Techniques and Expert Systems , Yasunari Matsuzaka, Ryu Yashiro , Published in MDPI AI Journal , 2023

- [6] Hand Gesture Recognition Using Convolutional Neural Networks , Vijay Madaan, Neha Sharma , Published in Chitkara University Journal of Engineering , 2023

- [7] Hand Sign & Gesture Recognition System , Priyal Raj, M. Singh, S. Pandey, Y. Singh ,Published in IEEE Access , 2023

- [8] Hand Gesture Based AI Controller for Presentation, Virtual Drawing and System Volume Management , Akshay Kumar, H. Anandaram, R. Pandey, K. Joshi, K. Alam, S. Chaudhary , Published by Amrita Vishwa Vidyapeetham / GEHU / Uttaranchal University , 2023

- [9] Gesture Based Model Using Deep Learning , Anuja Dixit, S. Jain, A. Kazmi, Aakanksha, K. Manorama , Published in Ajay Kumar Garg Engineering College Journal , 2023

- [10] Efficient Gesture-Based Presentation Controller Using Transfer Learning Algorithm , Shaik Faiz Ahamed, S. Srithar, P. Sandeep, T. Paraselli , Published in IEEE ICCCI Conference , 2023

- [11] Dynamic Hand Gesture Recognition , Subash Chandra Bose Jaganathan, K. Basak, K. R., S. Verma, T. Prakash, A. Mital , Published in VIT Bhopal Journal of Engineering , 2023
- [12] Smart Presentation System Using Hand Gestures , Bhairavi Pustode, V. Pawar, T. Pawar, S. Pokale , Published in Research Square , 2023
- [13] Controlling PowerPoint Presentation Using Hand Gestures in Real-Time , C. S. Charan, K. Meenakshi, V. B. Reddy, V. Kashyap , Published in IEEE Access , 2023
- [14] Automated Digital Presentation Control Using Hand Gesture Technique ,Salonee Powar, S. Kadam, S. Malage, P. Shingane , Published in IEEE ICACC Conference , 2022
- [15] Hand Gesture Recognition Based on Computer Vision: A Review of Techniques , Munir Oudah, A. Al-Naji, J. Chahl , Published in MDPI Journal of Imaging ,2020

